

Situation n° 2/2 :

Application mobile de gestion commerciale

Flutter / Dart / API REST / MySQL

PME Impression Personnalisée – Montpellier – Complément mobile de la situation 1/2

Table des matières

Le contexte	3
Le client	3
Le réseau de l'entreprise.....	4
La réalisation	4
Les fonctionnalités	5
La démarche.....	6
La base de données + UML	7
Quelques fonctionnalités expliquées.....	8
Authentification mobile sécurisée (JWT)	8
Consultation des fiches clients	9
Création de commande depuis le terrain.....	9
Suivi des commandes en temps réel.....	10
Notes terrain avec mode hors-ligne.....	10
Maquettes de l'application mobile	11
Serveur de versions	12
Axes d'amélioration.....	12
Conclusion	13

Le contexte

Dans le cadre de la modernisation de son système d'information, la PME spécialisée dans l'impression personnalisée a engagé une réflexion globale visant à améliorer sa gestion commerciale. Le premier axe d'amélioration, traité dans la situation 1/2, portait sur la mise en place d'un site web interne de gestion commerciale (PHP / MySQL / Bootstrap) permettant de centraliser les données.

Cependant, cette solution, bien qu'essentielle pour la gestion quotidienne en interne, présente une limite majeure : elle est exclusivement accessible depuis les postes de travail fixes de l'entreprise. Or, les commerciaux ne travaillent plus uniquement depuis les locaux. Ils se déplacent régulièrement pour rencontrer leurs clients, prospecter et assurer le suivi des commandes sur le terrain.

Ces déplacements génèrent plusieurs problèmes opérationnels concrets que le site web seul ne peut résoudre : accès impossible aux informations clients pendant les rendez-vous, prise de notes manuelle avec risque d'oubli et d'erreur, ressaisie des données au retour au bureau, incapacité à créer une commande immédiatement après un accord client. Ces contraintes impactent directement la qualité du suivi commercial et la réactivité de l'entreprise.

C'est dans ce contexte qu'intervient la présente situation professionnelle : le développement d'une application mobile dédiée aux commerciaux, complémentaire au site web, permettant un accès aux données commerciales et une saisie directe depuis le terrain.

Le client

L'entreprise cliente est la même PME spécialisée dans l'impression personnalisée que dans la situation 1/2 (Gestiolmprim, Montpellier). Pour rappel, elle propose des cartes de visite, flyers, affiches et textiles personnalisés à une clientèle de professionnels locaux.

Dans le cadre de cette situation 2, les utilisateurs principaux de l'application mobile sont les commerciaux de l'entreprise. Leur profil d'utilisation est spécifique :

- Déplacements fréquents chez les clients (1 à 5 rendez-vous par jour)
- Utilisation sur smartphone Android personnel ou professionnel
- Connexion internet variable selon la zone géographique (4G, WiFi client)
- Besoin de rapidité et de simplicité : pas de formation technique préalable
- Consultation des fiches clients et historiques pendant les rendez-vous

Problématique identifiée : Comment permettre aux commerciaux d'accéder aux données commerciales et de saisir des informations en déplacement, tout en garantissant la cohérence des données avec le site web interne ?

Le réseau de l'entreprise

L'architecture technique de l'application mobile s'articule autour de trois composants principaux qui s'intègrent au système d'information existant (site web de la situation 1).



La base de données MySQL est unique et partagée entre le site web (situation 1) et l'API REST utilisée par l'application mobile. Cela garantit une cohérence totale des données entre les deux outils : une commande créée depuis le mobile est immédiatement visible depuis le site web du service administratif, et inversement.

La sécurité des échanges entre l'application et l'API est assurée par des tokens JWT (JSON Web Token) avec une durée de validité de 24 heures. En cas de connexion intermittente, un système de synchronisation différée permet de stocker temporairement les notes terrain en local (SQLite) et de les synchroniser dès le retour en ligne.

La réalisation

Notre réalisation consiste à concevoir et développer une application mobile native Android, complémentaire au site web de gestion commerciale (situation 1). Elle offre aux commerciaux un accès simplifié aux données essentielles depuis leur smartphone, en situation de mobilité.

L'objectif n'est pas de reproduire l'intégralité du site web sur mobile, mais de proposer un ensemble de fonctionnalités ciblées et adaptées à un usage terrain : consultation rapide des fiches clients, création de commandes lors des rendez-vous, suivi des statuts et prise de notes contextuelles.

Technologie	Usage	Justification
Flutter 3 (Dart)	Développement de l'application mobile Android	Cross-platform, performances natives, large communauté, UI Material Design
PHP Lumen (API REST)	Exposition des données via endpoints HTTP/JSON	Framework PHP léger, idéal pour les API REST, cohérence avec situation 1
JWT (Auth)	Sécurisation des requêtes API	Stateless, sécurisé, adapté aux applications mobiles
SQLite (local)	Stockage temporaire des notes hors connexion	Base de données embarquée, support natif Flutter via sqflite
BLoC Pattern	Architecture de l'application Flutter	Séparation UI / logique métier, testabilité, maintenabilité

L'architecture BLoC (Business Logic Component) adoptée pour l'application Flutter garantit une séparation claire entre l'interface utilisateur, la logique métier et les appels API. Chaque fonctionnalité dispose de son propre BLoC, qui reçoit des événements de l'UI, interagit avec les services et émet des états que l'UI affiche.

Les fonctionnalités

L'application mobile de gestion commerciale propose les fonctionnalités suivantes :

Authentification sécurisée par token JWT

Connexion par identifiant/mot de passe. Le token JWT reçu est stocké de manière sécurisée via flutter_secure_storage et envoyé dans le header Authorization de chaque requête. Session mémorisée pour éviter une reconnexion à chaque démarrage.

Consultation des fiches clients

Recherche et consultation des fiches clients depuis le smartphone. Chaque fiche affiche les coordonnées, l'historique des commandes et les notes terrain associées. Optimisée pour une utilisation rapide lors des rendez-vous.

Recherche multicritères instantanée

Recherche de clients et commandes par nom, email, téléphone ou référence. Résultats affichés en temps réel via l'API REST. Interface épurée adaptée aux petits écrans.

Création de commande directement sur le terrain

Sélection du client, ajout de produits du catalogue avec quantités, calcul automatique du total. La commande est soumise à l'API et immédiatement visible depuis le site web interne. Confirmation avant validation pour éviter les erreurs.

Suivi en temps réel des commandes

Consultation du statut de toutes les commandes avec code couleur. Mise à jour possible du statut depuis l'application mobile par les commerciaux autorisés. Permet d'informer le client en direct lors des rendez-vous.

Notes terrain avec mode hors-ligne

Saisie de notes contextuelles rattachées à un client, horodatées automatiquement. En l'absence de connexion, les notes sont sauvegardées en local (SQLite) et synchronisées dès le retour en ligne. Visible également depuis le site web.

Synthèse du cahier des charges :

Fonctionnalité	Technologies	Disponibilité offline
Authentification JWT	Flutter, PHP Lumen, JWT	Session mémorisée
Consultation / recherche clients	Flutter, HTTP, JSON	✗ Connexion requise
Création de commandes	Flutter, API REST	✗ Connexion requise
Suivi des commandes	Flutter, HTTP, JSON	✗ Connexion requise
Notes terrain	Flutter, SQLite, API	✓ Mode hors-ligne
Historique client	Flutter, API REST	✗ Connexion requise

La démarche

Pour mener à bien ce projet, nous avons adopté une méthodologie de développement itérative et incrémentale, en continuité avec l'approche utilisée pour le site web (situation 1). Des validations régulières ont été organisées avec les commerciaux utilisateurs finaux.

Analyse des besoins terrain

Des entretiens spécifiques ont été menés avec les commerciaux pour comprendre leurs usages réels : type d'appareils utilisés, fréquence des déplacements, situations de connexion intermittente, besoins prioritaires. Ces observations ont orienté les choix de conception de l'interface.

Développement de l'API REST

Avant le développement de l'application mobile, une API REST a été développée en PHP (Lumen) pour exposer les données de la base MySQL existante. Chaque fonctionnalité de l'application correspond à un ou plusieurs endpoints REST sécurisés par JWT.

Développement Flutter par fonctionnalité

Le développement a été réalisé fonctionnalité par fonctionnalité en suivant le pattern BLoC : définition des événements et états, développement du BLoC, développement des services API correspondants, puis développement des écrans Flutter. Des tests unitaires ont été réalisés sur les BLoC.

Tests sur appareils réels

Les tests ont été réalisés directement sur les smartphones Android des commerciaux pour valider l'ergonomie en conditions réelles. Des scénarios de connexion intermittente ont été simulés pour tester le mode hors-ligne des notes terrain.

Le planning prévisionnel :

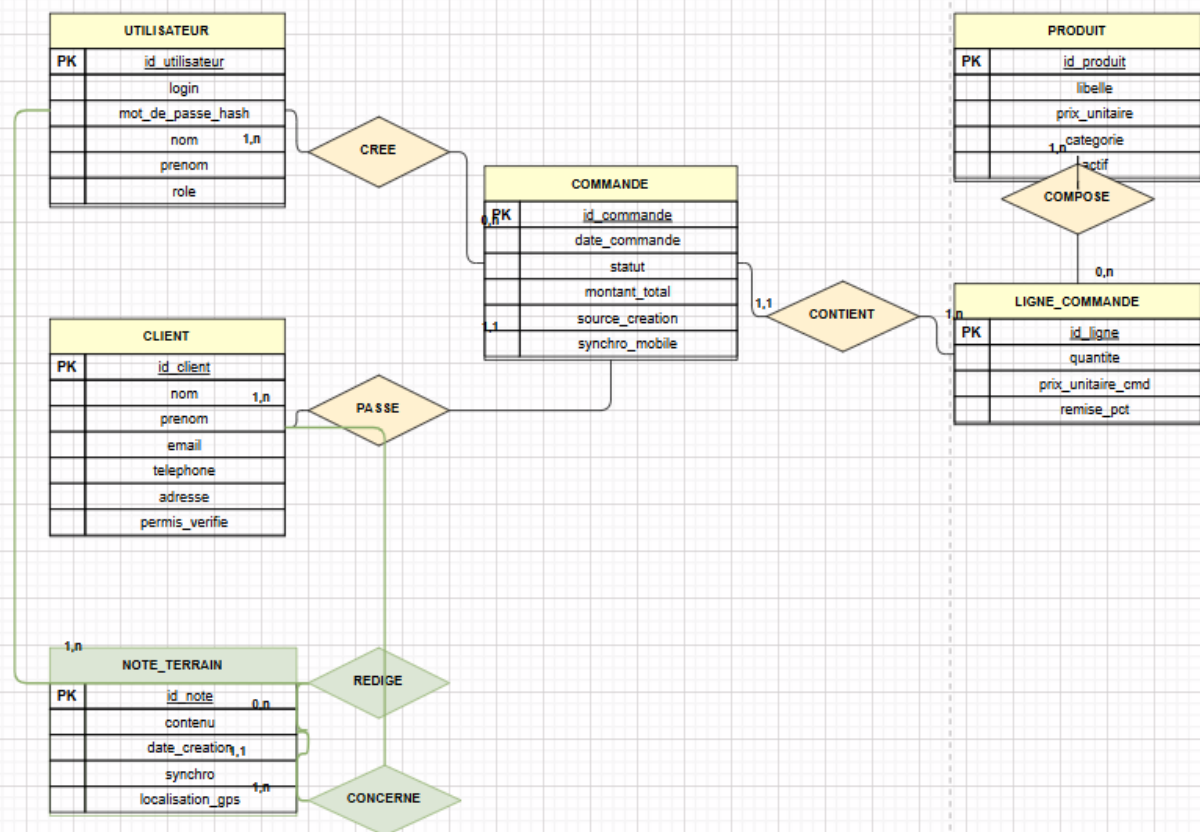
Étape	Durée	Livrable
Analyse besoins terrain + entretiens commerciaux	Quelques jours	Cahier des charges mobile
Conception MCD enrichi + UML + maquettes	1 semaine	MCD, diagrammes, maquettes mobile
Développement API REST PHP (Lumen)	1 semaine	API REST documentée
Développement application Flutter	4 semaines	Application mobile fonctionnelle
Tests fonctionnels & BLoC unitaires	En continu	Rapport de tests
Tests sur appareils réels (commerciaux)	1 semaine	PV de recette
Déploiement APK + formation commerciaux	2 jours	APK + guide utilisateur

La base de données + UML

Modèle Conceptuel de Données (MCD)

Le MCD de l'application mobile enrichit le modèle de données existant (situation 1) avec une nouvelle entité : NOTE_TERRAIN. Cette entité, propre à l'usage mobile, permet aux commerciaux de consigner des informations lors de leurs déplacements. La table COMMANDE est également

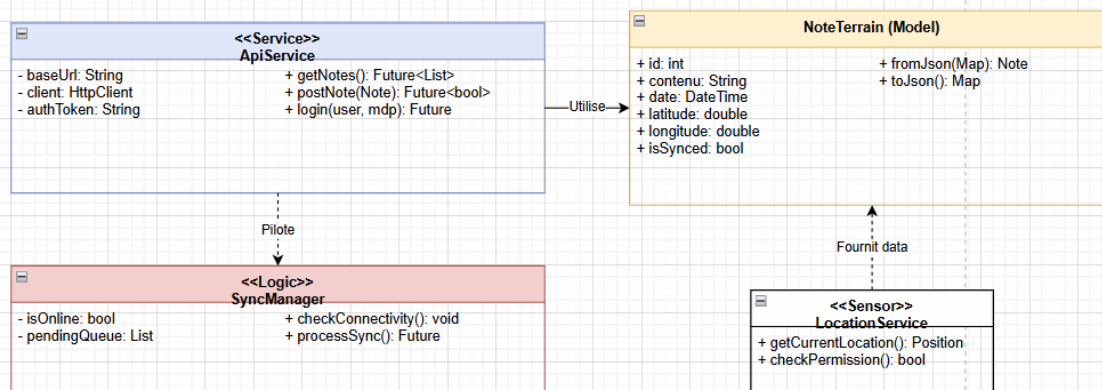
enrichie avec l'identifiant de l'utilisateur ayant créé la commande.



MCD – Situation 2 : Application mobile de gestion commerciale (Flutter / API REST / MySQL)

Diagramme de classes UML (modèles Dart)

Le diagramme de classes UML représente la structure des modèles Dart de l'application Flutter ainsi que le service API principal. Chaque classe modèle correspond à une entité de la base de données et dispose de méthodes de sérialisation JSON pour les échanges avec l'API.



Endpoints API REST (extrait)

```
# API REST - Endpoints principaux (PHP Lumen)

POST    /api/auth/login      → Authentification, retourne le JWT
POST    /api/auth/logout    → Invalidation du token

GET      /api/clients        → Liste des clients (filtrable ?q=recherche)
GET      /api/clients/{id}   → Détail d'un client
POST     /api/clients        → Créer un client
PUT      /api/clients/{id}   → Modifier un client

GET      /api/commandes      → Liste des commandes (?statut=&id_client=)
GET      /api/commandes/{id} → Détail commande + lignes
POST     /api/commandes      → Créer une commande (avec lignes)
PATCH   /api/commandes/{id} → Modifier le statut

GET      /api/notes          → Notes terrain de l'utilisateur connecté
POST     /api/notes          → Créer une note (sync offline)
POST     /api/notes/sync     → Synchronisation groupée (offline-online)
```

Quelques fonctionnalités expliquées

Authentification mobile sécurisée (JWT)

Libellé de la fonctionnalité : Authentification par token JWT avec stockage sécurisé

Explication de la fonctionnalité : Les commerciaux se connectent via un écran simple (identifiant / mot de passe). L'application envoie ces informations à l'endpoint `/api/auth/login` de l'API REST. Après vérification en base de données et hashage bcrypt, l'API retourne un JWT signé valable 24 heures. Ce token est stocké localement de manière sécurisée via `flutter_secure_storage` (Keychain iOS / Keystore Android). Il est ensuite automatiquement ajouté au header `Authorization` de toutes les requêtes suivantes.

Extrait de code – Service d'authentification (Dart/Flutter)

```
// auth_service.dart - Authentification et gestion du token JWT
import 'dart:convert';
import 'package:http/http.dart' as http;
import 'package:flutter_secure_storage/flutter_secure_storage.dart';

class AuthService {
  final _storage = const FlutterSecureStorage();
  final String _baseUrl = 'https://api.gestiimprim.fr';

  Future<bool> login(String login, String password) async {
    final response = await http.post(
      Uri.parse('${_baseUrl}/api/auth/login'),
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode({'login': login, 'password': password}),
    );
    if (response.statusCode == 200) {
      final data = jsonDecode(response.body);
      // Stockage sécurisé du token JWT
      await _storage.write(key: 'jwt_token', value: data['token']);
      await _storage.write(key: 'user_nom', value: data['user']['nom']);
      return true;
    }
    return false; // Identifiants invalides
  }
}
```



```
Future<String?> getToken() => _storage.read(key: 'jwt_token');
}
```

Consultation des fiches clients

Libellé de la fonctionnalité : Recherche et consultation des clients en mobilité

Explication de la fonctionnalité : L'écran de consultation clients affiche la liste des clients avec une barre de recherche en temps réel. La recherche est envoyée à l'API avec un délai de 300ms (debounce) pour éviter de surcharger le serveur. En appuyant sur un client, l'utilisateur accède à sa fiche complète : coordonnées, adresse, nombre de commandes, dernière commande et notes terrain associées. L'interface est épurée et optimisée pour une utilisation rapide lors des rendez-vous.

Extrait de code – BLoC de recherche client (Dart)

```
// client_bloc.dart - Gestion de la recherche client avec BLoC
class ClientBloc extends Bloc<ClientEvent, ClientState> {
  final ClientRepository _repo;
  Timer? _debounce;

  ClientBloc(this._repo) : super(ClientInitial()) {
    on<ClientSearchChanged>((event, emit) {
      _debounce?.cancel();
      _debounce = Timer(const Duration(milliseconds: 300), () {
        add(ClientSearchSubmitted(event.query));
      });
    });
    on<ClientSearchSubmitted>((event, emit) async {
      emit(ClientLoading());
      try {
        final clients = await _repo.search(event.query);
        emit(ClientLoaded(clients));
      } catch (_) { emit(ClientError()); }
    });
  }
}
```

Création de commande depuis le terrain

Libellé de la fonctionnalité : Saisie de commande lors d'un rendez-vous client

Explication de la fonctionnalité : Le flux de création de commande se déroule en 3 étapes dans l'application : (1) sélection du client parmi la liste, (2) ajout des produits du catalogue avec quantités et calcul automatique du total, (3) validation avec confirmation. La commande est envoyée à l'API REST sous forme d'un objet JSON contenant les informations de la commande et la liste des lignes. Après validation, la commande est immédiatement visible depuis le site web du service administratif, sans ressaisie nécessaire.

Extrait de code – Envoi de commande à l'API (Dart)

```
// commande_service.dart - Création de commande via API REST
Future<bool> createCommande(CommandeCreate cmd) async {
  final token = await _storage.read(key: 'jwt_token');
  final response = await http.post(
    Uri.parse('${_baseUrl}/api/commandes'),
    headers: {
      'Authorization': 'Bearer $token',
      'Content-Type': 'application/json',
    },
  );
}
```

```

    },
    body: jsonEncode({
      'id_client': cmd.idClient,
      'statut': 'attente',
      'lignes': cmd.lignes.map((l) => {
        'id_produit': l.idProduit,
        'quantite': l.quantite,
        'prix_unitaire': l.prixUnitaire,
      }).toList(),
    }),
  );
  return response.statusCode == 201; // HTTP 201 Created
}

```

Suivi des commandes en temps réel

Libellé de la fonctionnalité : Consultation et mise à jour du statut des commandes

Explication de la fonctionnalité : L'écran de suivi des commandes affiche la liste des commandes avec un code couleur par statut : rouge (En attente), orange (En cours), vert (Livrée), gris (Annulée). Des filtres permettent d'afficher uniquement les commandes d'un client ou d'un statut spécifique. Le commercial peut consulter le détail d'une commande et, selon son rôle, mettre à jour son statut via une requête PATCH à l'API. Cette mise à jour est immédiatement répercutée sur le site web interne, permettant une coordination en temps réel entre les équipes.

Extrait de code – Mise à jour du statut (Dart)

```

// commande_service.dart - PATCH statut d'une commande
Future<bool> updateStatut(int idCommande, String newStatut) async {
  final token = await _storage.read(key: 'jwt_token');
  final response = await http.patch(
    Uri.parse('${_baseUrl}/api/commandes/$idCommande'),
    headers: {'Authorization': 'Bearer $token', 'Content-Type': 'application/json'},
    body: jsonEncode({'statut': newStatut}),
  );
  return response.statusCode == 200;
}

```

Notes terrain avec mode hors-ligne

Libellé de la fonctionnalité : Prise de notes avec synchronisation différée

Explication de la fonctionnalité : Le commercial peut saisir des notes contextuelles lors de ses rendez-vous, rattachées à un client. Si la connexion internet est disponible, la note est directement envoyée à l'API et enregistrée en base. En l'absence de connexion (zone blanche, tunnels...), la note est sauvegardée localement dans SQLite (SQLite) avec un flag synchro=false. À chaque reprise de connexion, l'application détecte automatiquement les notes en attente et les synchronise via l'endpoint /api/notes/sync. Cette approche garantit que aucune information n'est perdue, même sans connexion.

Extrait de code – Sauvegarde offline et synchronisation (Dart)

```

// notes_service.dart - Gestion offline avec SQLite
Future<void> saveNote(Note note) async {
  final connected = await _connectivity.checkConnectivity() !=
  ConnectivityResult.none;
  if (connected) {
    await _postToApi(note); // Envoi direct
  }
}

```

```

    } else {
        // Stockage local en attente de synchronisation
        await _localDb.insert('notes_pending', {
            'contenu': note.contenu,
            'id_client': note.idClient,
            'date_creation': note.dateCreation.toIso8601String(),
            'synchro': 0,
        });
    }
}

// Appelée automatiquement au retour en ligne
Future<void> syncPendingNotes() async {
    final pending = await _localDb.query('notes_pending', where: 'synchro = 0');
    if (pending.isEmpty) return;
    await _postBulkToApi(pending); // Envoi groupé
    await _localDb.update('notes_pending', {'synchro': 1}, where: 'synchro = 0');
}

```

Maquettes de l'application mobile

Les maquettes suivantes représentent les principaux écrans de l'application mobile tels qu'ils ont été conçus lors de la phase de conception. Elles ont été validées avec les commerciaux lors des sessions de travail préliminaires.

Écran de Connexion



GestioImprim
Application commerciale

Identifiant

jean.dupont

Mot de passe

.....

SE CONNECTER

— Réseau sécurisé —

Écran d'Accueil – Tableau de bord

Bonjour, Jean Dupont

14 Mes commandes	47 Clients
3 En attente	7 Notes terrain

Actions rapides :

- Rechercher un client →
- + Nouvelle commande →
- Prendre une note →

Écran – Liste des Clients

Rechercher... **OK**

Paul DUPONT 06 12 34 56 78 · 3 commandes	Voir
Sophie MARTIN 07 98 76 54 32 · 1 commande	Voir
André LEFEBVRE 06 55 44 33 22 · 5 commandes	Voir

+ Nouveau client

Écran – Nouvelle Commande

← Retour

Nouvelle commande

Client sélectionné :

Paul DUPONT – 06 12 34 56 78

Produits :

Produit	Qté	Total
Cartes de visite 500ex	2	90 €

Flyers A5 r/v	1	120 €
+ Ajouter		
	TOTAL	210 €
✓ VALIDER LA COMMANDE		

Serveur de versions

Le projet a été versionné avec Git, hébergé sur GitHub en monorepo regroupant le code de l'API REST et de l'application Flutter. Cette organisation facilite la cohérence entre les deux composants et simplifie le déploiement.

- Dépôt GIT : <https://github.com/delplace-martin/appli-mobile-commerciale>
- Structure : /api (PHP Lumen) + /mobile (Flutter/Dart)
- Stratégie de branches : main / develop / feature-[nom]
- Commits réguliers avec convention : feat:, fix:, api:, mobile:

Axes d'amélioration

À l'issue du développement, plusieurs axes d'amélioration ont été identifiés pour les évolutions futures :

- Portage iOS de l'application (Flutter permet un code source unique pour Android et iOS)
- Mode hors-ligne étendu : synchronisation complète des fiches clients et commandes, pas seulement les notes
- Intégration d'un module de géolocalisation pour visualiser les clients sur une carte interactive
- Notifications push via Firebase Cloud Messaging : alertes de changement de statut de commande
- Module de signature électronique pour valider les commandes directement sur l'appareil du client
- Tableau de bord mobile personnalisé avec les indicateurs de performance du commercial
- Intégration de la reconnaissance vocale pour la saisie des notes terrain

Conclusion

Le développement de cette application mobile constitue un complément indispensable au site web de gestion commerciale (situation 1). Ensemble, ces deux outils forment un système d'information complet et cohérent : le site web assure la gestion interne centralisée, tandis que l'application mobile étend ces fonctionnalités aux situations de mobilité des commerciaux.

Les bénéfices apportés à l'entreprise sont concrets et mesurables : suppression des ressaisies manuelles et des erreurs associées, création de commandes immédiate lors des rendez-vous, accès à l'historique client pendant les échanges, prise de notes contextuelles sans risque d'oubli, et meilleure réactivité commerciale globale.

La stack technique Flutter / Dart / API REST / MySQL choisie s'est révélée particulièrement adaptée : Flutter permet de développer une application native performante à partir d'un code source unique, l'API REST garantit la cohérence des données avec le site web existant, et le pattern BLoC assure une architecture maintenable et testable.

Sur le plan des compétences, cette réalisation m'a permis d'acquérir et de mettre en pratique des compétences complémentaires à celles développées en situation 1 : développement mobile natif avec Flutter/Dart, conception et développement d'une API REST sécurisée par JWT, gestion du mode hors-ligne avec SQLite, et prise en compte des contraintes spécifiques à l'usage mobile (ergonomie, performances, connectivité intermittente).